Experiment 9 Design and Demonstration of Test Cases

DINIECO Smart Dining Pad web application

Functional testing, non-functional testing, execution evidence, and defect analysis

Submission field	Value
Student name	[Enter student name]
Register number	[Enter register number]
Institution	[Enter institution]
Faculty	[Enter faculty name]
Experiment date	18 October 2026

Execution summary: 20 functional cases passed and 1 failed. Five saved Selenium cases passed in Chrome through the official SIDE runner; the Chrome extension GUI was blocked by organization account access. JMeter completed 454 local read-only requests with no errors.

2 Experiment number and title

Exp-9: Design and Demonstration of Test Cases. Application under test: DINIECO Smart Dining Pad, Flutter Web frontend and FastAPI backend.

3 Objective

Design traceable functional and non-functional test cases; execute feasible tests against the existing application; preserve genuine evidence; identify defects; and report measured outcomes and limits.

4 Introduction to functional and non-functional testing

Functional testing checks whether specific behaviors meet expected results, such as authentication rejection or input validation. Non-functional testing measures qualities such as response time, latency, throughput, and error rate under a defined workload. A saved test definition is not an execution result.

5 Application under test

The DINIECO app provides account registration/login, a dashboard, meal sessions, image upload and food analysis, nutrition estimates, waste analytics, and smart-pad diagnostics. The frontend uses Flutter 3.47.5 and Dart 3.13.4; the API uses FastAPI, SQLAlchemy and SQLite. The clean browser launch reached the Sign In page. The local backend responded at http://127.0.0.1:8000 and the web build at http://127.0.0.1:8080/.

The current pad driver is an in-memory simulator. The observed analysis response used a deterministic catalog fallback with ai_verified=false. No verified live Gemini inference or physical BLE pad connection is claimed.

6 Software and hardware requirements

Item	Experiment environment
Operating environment	Windows local development machine
Web runtime	Google Chrome 155.0.8059.84
Frontend	Flutter 3.47.5, Dart 3.13.4, local static server on port 8080
Backend	Python 3.13, FastAPI/Uvicorn on port 8000, SQLite
Load tool	Java 21.0.12.1, Apache JMeter 5.6.3
Physical Smart Pad	Not verified or connected

7 Tools used and versions

Chrome 155.0.8059.84 rendered browser evidence. Selenium SIDE runner 4.0.13 executed the five saved .side cases in Chrome; the extension GUI could not be installed because the signed-in organization account was denied Chrome Web Store access. Apache JMeter 5.6.3 ran the load plan with Java 21.0.12.1. Python HTTP probes, backend pytest and Flutter tests supplied supplementary evidence.

8 Test environment and configuration

All requests targeted localhost. The existing backend on port 8000 and Flutter release build on port 8080 were used. Authenticated tests created synthetic local accounts and uploaded a synthetic JPEG; identifiers, passwords and tokens are excluded from shared evidence. The JMeter plan targeted only GET /, GET /health and GET /api/v1/device/status. No write, upload, AI or hardware command received load.

9 Functional testing methodology

Requirements were mapped to independent cases. The saved Selenium IDE-format project ran through the official SIDE command line runner in Chrome. Separate authenticated Selenium WebDriver sessions exercised login, the active meal screen, synthetic image selection, preview and upload. Live API requests checked status, validation and analysis

provenance. Source inspection established the device simulator and client defects. PASS denotes an executed assertion, FAIL an observed mismatch and BLOCKED an unexecuted dependency.

10 Test case table

The full reproducible table, including preconditions, data, steps, expected and actual results, evidence filenames and timestamps, is in test_case_table.csv. The compact summary below retains all 21 case IDs.

ID	Scenario	Actual result	Status
WEB-01	Unauthenticated initial page	Sign In page rendered in Chrome	PASS
API-01	Platform root	HTTP assertion passed; response saved	PASS
API-02	Backend and database health	HTTP assertion passed; response saved	PASS
API-03	Anonymous profile rejected	HTTP assertion passed; response saved	PASS
API-04	Invalid login rejected	HTTP assertion passed; response saved	PASS
API-05	Invalid zero food weight rejected	HTTP assertion passed; response saved	PASS
API-06	Device status schema	HTTP assertion passed; response saved	PASS
API-07	Weight stream schema	HTTP assertion passed; response saved	PASS
API-08	Anonymous upload rejected	HTTP assertion passed; response saved	PASS
SRC-01	Distinguish simulated pad data from real sensor data	API claims connected; source uses fixed simulator values	FAIL
SEL-01	IDE initial auth screen	SIDE runner assertion passed in Chrome	PASS
SEL-02	IDE health response	SIDE runner assertion passed in Chrome	PASS
SEL-03	IDE anonymous access	SIDE runner assertion passed in Chrome	PASS
SEL-04	IDE invalid login	SIDE runner assertion passed in Chrome	PASS
SEL-05	IDE empty login validation	SIDE runner assertion passed in Chrome	PASS
WEB-02	Authenticated photo upload and preview	Browser preview and backend upload passed	PASS
WEB-03	Live AI analysis and provenance	500 g catalog result; ai_verified=false	PASS
API-09	Missing image file validation	HTTP 422 with file field required	PASS
API-10	Unsupported image type validation	HTTP 400; only JPEG, PNG and WebP accepted	PASS
API-11	Empty image validation	HTTP 400 Uploaded file is empty	PASS
WEB-04	No fabricated meal before photo	Awaiting Meal Photo and 0 g before upload	PASS

11 Selenium IDE test execution results

The official Selenium SIDE runner 4.0.13 executed all five saved .side tests in Chrome 155: 5 passed, 0 failed. Initial runner runs exposed incorrect title/text assertions and Flutter semantics locators; the test project was corrected and rerun. The final result JSON is results-2026-10-18T05-47-46.485Z.json. The Chrome Web Store denied the signed-in organization account, so the extension GUI could not be installed. These are runner results, not an IDE GUI recording or playback demonstration.

12 Functional execution screenshots

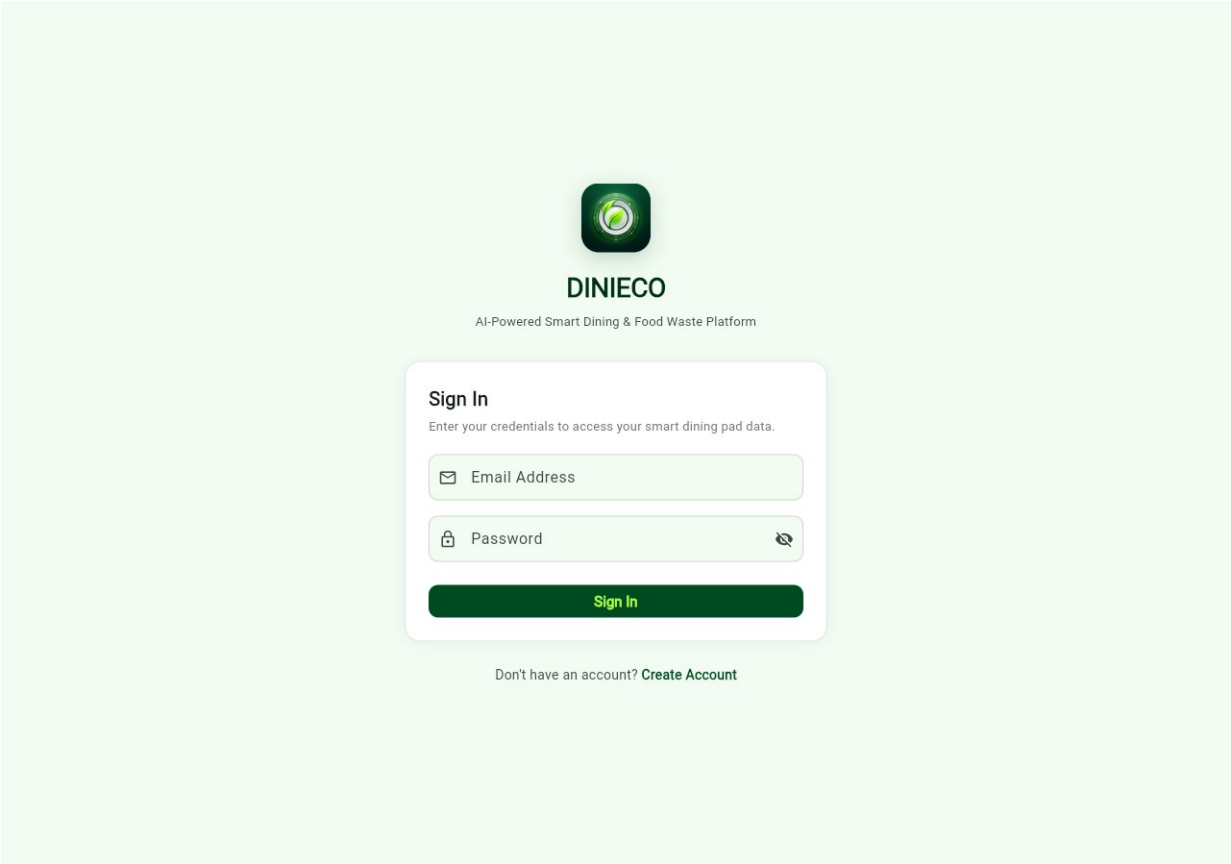


Figure 1. Genuine Chrome headless rendering of the unauthenticated DINIECO Sign In page, 18 October 2026.

Selenium SIDE runner · Chrome

Exp9_SmartDiningPad.side · official runner 4.0.13 · saved execution result

5 passed · 0 failed · 5 total

Test case	Result	Duration
SEL-01 Unauthenticated app shell	PASSED	3.78 s
SEL-02 Backend health response	PASSED	1.73 s
SEL-03 Protected profile rejects anonymous visitor	PASSED	1.62 s
SEL-04 Invalid login validation	PASSED	5.09 s
SEL-05 Empty login validation	PASSED	4.91 s

Source: results-2026-10-18T05-47-46.485Z.json. This is a visual summary of the genuine runner JSON, not an extension GUI screenshot.

Figure 2. Selenium SIDE runner visual summary generated from the final five-pass JSON result.

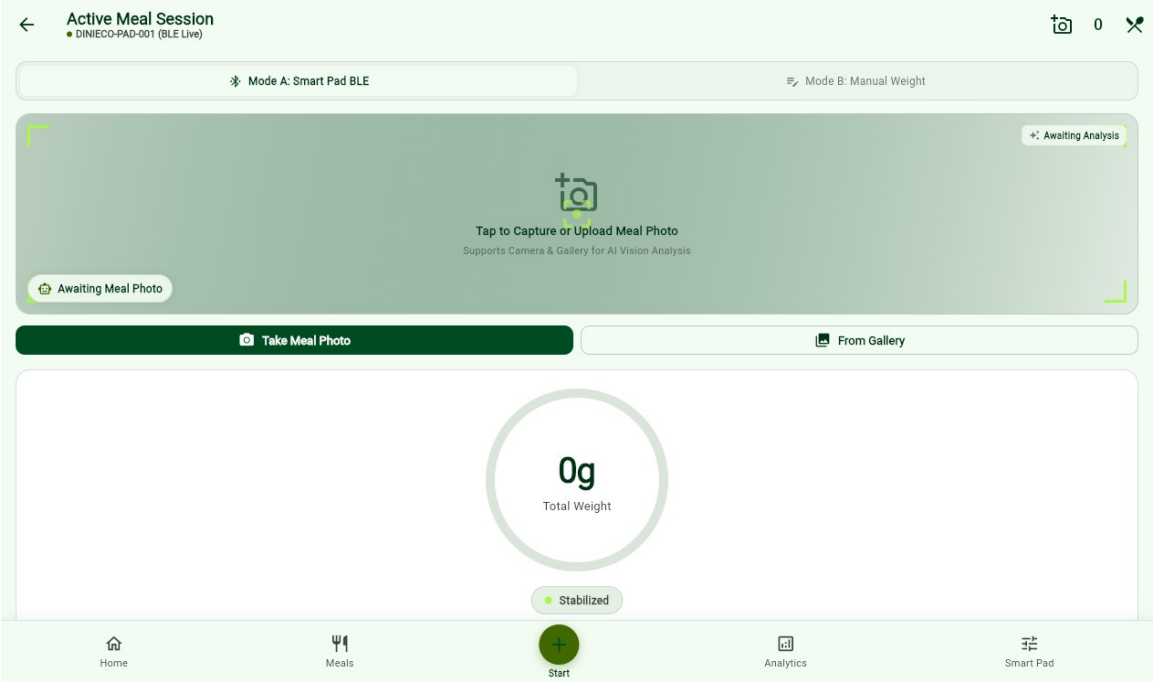


Figure 3. Authenticated active session before selecting a photo: Awaiting Meal Photo and 0 g.

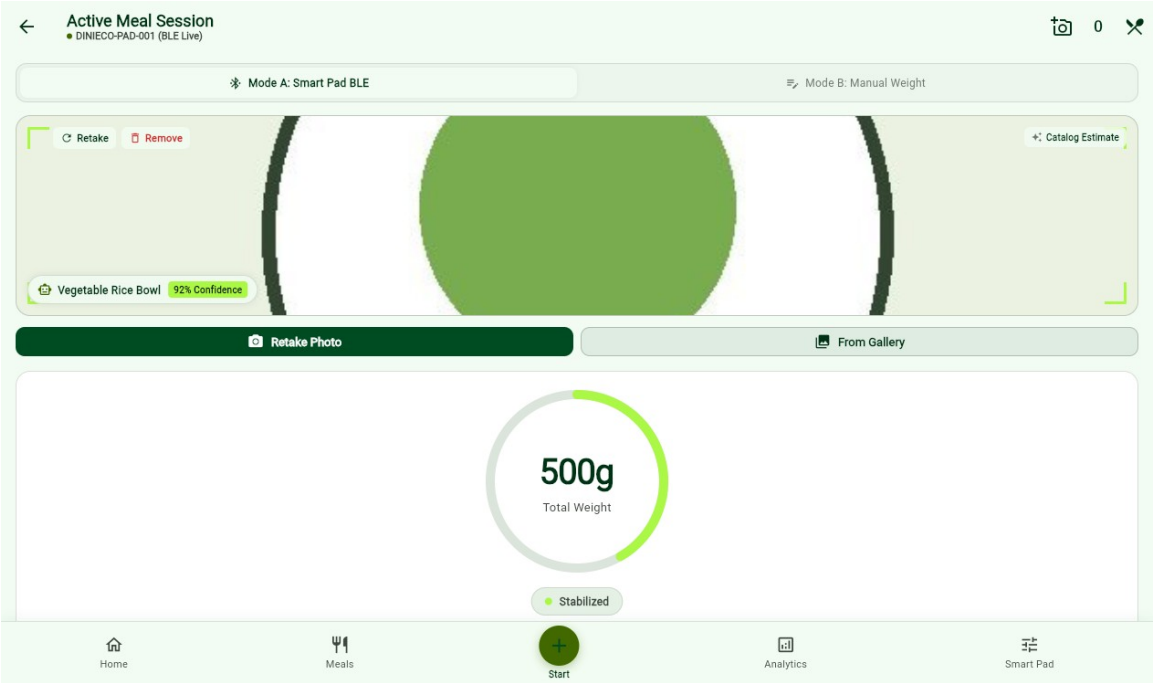


Figure 4. Synthetic JPEG preview and 500 g catalog estimate after authenticated upload and analysis.

13 Defect log and retesting

ID	Finding	Severity	State
D-01	Simulated pad status is presented as live hardware	High	Unresolved
D-02	Preloaded and error fallback meal values appeared as analysis	Medium	Fixed; browser retest passed
D-03	Web image upload used unsupported dart:io multipart path	High	Fixed; browser retest passed
D-04	Image analysis client timed out before usable backend result	Medium	Fixed; browser retest passed

D-01 remains unresolved: the API reports connected=true and 94% battery from an in-memory simulator without a verified BLE link. D-02's fixed default meal and error fallback were removed; the browser now shows an empty meal state before upload. D-03's dart:io multipart upload failed on web and was replaced with an XFile byte upload. D-04's short image-analysis timeout was extended and the catalog fallback is labeled in the UI. The complete defect log records original observations, changes and retests.

14 Performance testing methodology

JMeter 5.6.3 was run in non-GUI mode against three verified read-only endpoints. The plan used 10 virtual users, a 30-second ramp-up, a 60-second scheduler duration, and a one-second constant timer before each request. HTTP 200 response assertions were attached to each sampler. A JTL file and generated HTML report were saved. This workload avoids database writes, authentication credentials, uploads, AI charges, and hardware commands.

15 JMeter test plan configuration

Setting	Value
Target	http://127.0.0.1:8000; host/port can be overridden by JMeter properties
Thread group	10 users; ramp 30 s; duration 60 s; loop continuously
Pacing	Constant Timer 1000 ms per request
Samplers	GET /, GET /health, GET /api/v1/device/status
Assertions	Response code equals 200 per sampler
Output	performance_results.jtl; HTML dashboard; Summary and Aggregate listeners

16 Performance testing results

Endpoint	n	Mean ms	Median ms	P90 ms	Max ms	Error %	req/s
Total	454	7.86	4.00	10.00	148	0.00	7.73
Device status	145	13.15	5.00	24.40	148	0.00	2.64
Health	154	5.58	4.00	7.50	129	0.00	2.67
Platform root	155	5.17	4.00	8.00	68	0.00	2.67

Overall: 454 samples, 454 successful and 0 failed. Minimum response time was 2 ms; maximum 148 ms. JTL mean latency was 7.77 ms. JMeter received 2.14 KB/s and sent 1.16 KB/s. Error percentage = 0 / 454 × 100 = 0.00%. Throughput is completed requests divided by elapsed sample interval, in requests per second.

17 JMeter execution screenshots

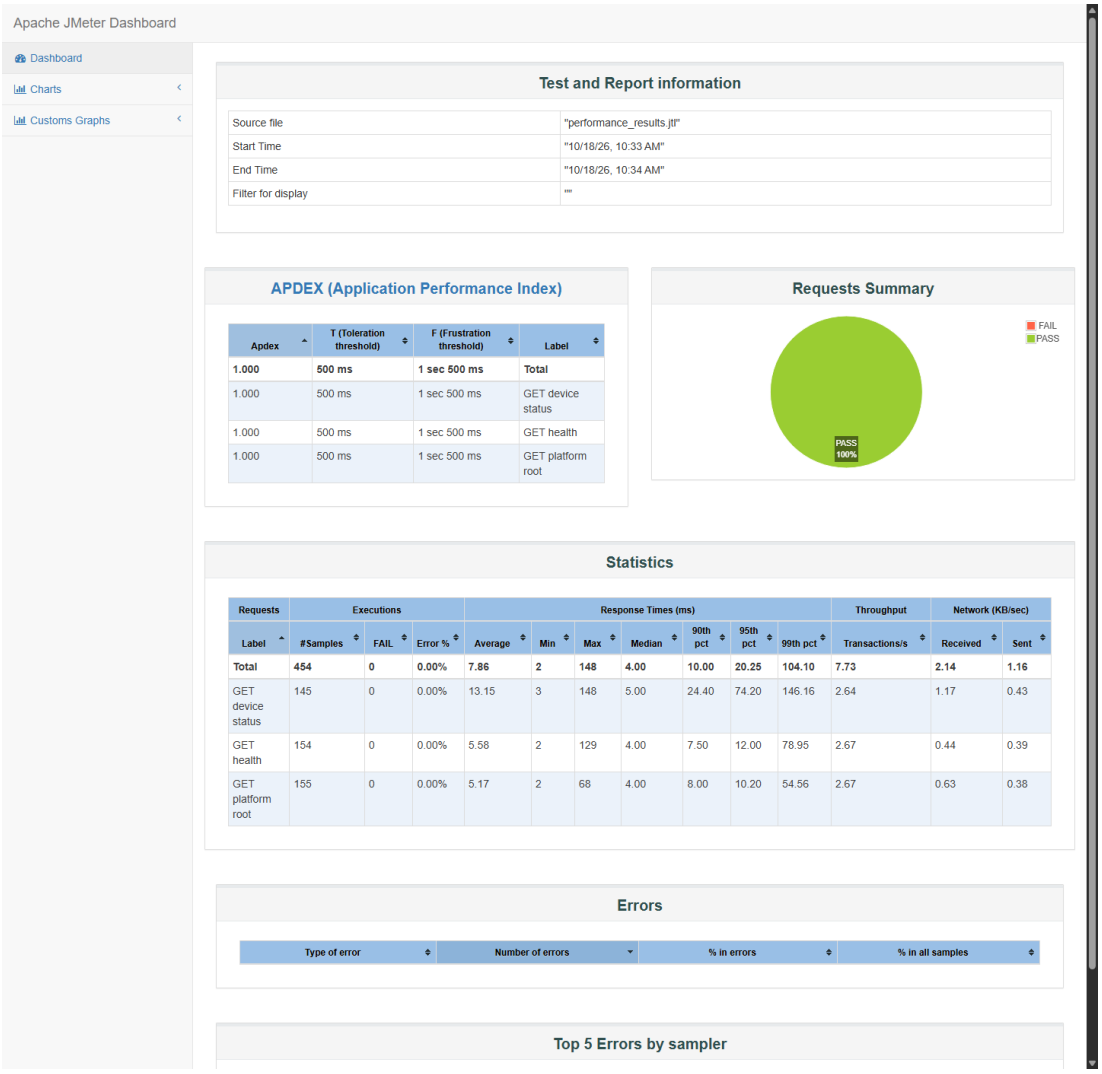


Figure 5. JMeter statistics table and zero-error summary from the completed 454-sample run.

Both JMeter HTML dashboard screenshots are included in the evidence folder. The figure shows the generated report; desktop Aggregate and Summary listener screenshots were not captured.

18 Analysis and discussion

Functional execution produced 20 passes and 1 failure among 21 executed cases, for a 95.24% executed-case pass rate. The five SIDE runner cases passed. Browser login, photo upload and empty-state retests passed. Direct analysis returned HTTP 200 with catalog-fallback and ai_verified=false; this is a catalog estimate, not verified live AI. Backend pytest reported 21 passes and Flutter reported 13 passes as supplementary evidence. Device provenance remains a material unresolved defect.

For the read-only local load, the 10.00 ms 90th percentile is higher than the 4.00 ms median; occasional requests reached 148 ms. Device status was the slowest sampler on average at 13.15 ms. No HTTP assertion failed. The run has no formal performance acceptance threshold and cannot establish behavior under upload, AI inference, database writes, or production traffic. The independent JTL recalculation matched JMeter's sample count, mean and throughput; its 9.7 ms interpolated P90 differs from JMeter's 10.00 ms percentile convention.

19 Limitations

The Chrome extension GUI could not be installed because the signed-in organization account lacks Chrome Web Store access; the official SIDE runner executed the .side suite instead. Synthetic test accounts and uploaded JPEGs remain in the local backend data; existing user records were not edited. The catalog fallback was observed, but a verified live Gemini result was not. Logout, persisted meal history and physical pad integration were not covered by the case set. The performance workload covered three inexpensive local GET endpoints for one 60-second run. The temporary runtime folder contains a database copy and test credentials and must be excluded from submission.

20 Conclusion

The experiment delivered 21 executed functional cases, a five-pass Selenium SIDE runner result in Chrome, authenticated browser upload evidence, four documented defects with three verified fixes, and a reproducible JMeter plan with measured results. Pad-status provenance remains the one observed failing case. A separate extension GUI demonstration still requires a Chrome account allowed to use the Web Store; no such GUI execution is claimed here.